

Praktikum 7: Suche in Graphen

Aufgabe 1: Breitensuche auf einer Höhenkarte

Sie erhalten eine Höhenkarte in Form einer ASCII-Darstellung. Jedes Zeichen entspricht einem Feld; der Buchstabe gibt die relative Höhe an (a = niedrigste, z = höchste Höhe). Das Zeichen S markiert Ihren Startpunkt (Höhe wie a), das Zeichen E das Ziel (Höhe wie z).

Von einem Feld aus können Sie in einem Schritt auf ein horizontal oder vertikal benachbartes Feld wechseln – jedoch nur dann, wenn das Zielfeld höchstens eine Höhenstufe höher liegt als das aktuelle. Ein Wechsel bergab oder auf gleicher Höhe ist immer erlaubt.

Das Beispielgelände:

```
Sabqponm  
abcryxxl  
accszExk  
acctuvwj  
abdefghi
```

Die Aufgabenstellung entspricht „Advent of Code“ von 2022, Tag 12.

(<https://adventofcode.com/2022/day/12>)

- Modellierung:** Modellieren Sie die Karte als gerichteten Graphen: Jedes Feld (Zeile, Spalte) ist ein Knoten; eine gerichtete Kante existiert von Feld A nach Feld B, wenn B ein horizontaler oder vertikaler Nachbar von A ist und die Bewegungsregel eingehalten wird. Welche Darstellung (Adjazenzliste oder Adjazenzmatrix) ist besser geeignet? Begründen Sie Ihre Wahl.
- Pfadsuche:** Bauen Sie den Graphen auf und nutzen Sie `bfs()` und `path()` aus `vorlesung/L08_graphen/graph.py`, um den kürzesten Pfad von S nach E zu finden. Geben Sie den Pfad und seine Länge aus.
- AoC:** Melden Sie sich bei „Advent of Code“ an und laden Sie Ihre eigene Karte herunter. Ermitteln Sie den kürzesten Pfad für die echte Karte. Messen Sie dabei die Laufzeit der BFS auf dem Beispielgelände und auf der echten Karte (z. B. mit `time.perf_counter()`). Vergleichen Sie die gemessenen Laufzeiten mit dem Verhältnis der jeweiligen $|V| + |E|$ -Werte beider Karten — bestätigt die Messung das lineare Wachstum $O(|V| + |E|)$?
- (Optional)** Nach dem Lösen der ersten Teilaufgabe gibt es eine zweite Teilaufgabe: Finden Sie den kürzesten Pfad von einem beliebigen Feld mit Höhe 'a' (oder S) zum Ziel E. Wie können Sie dieses Problem auf eine einzelne BFS zurückführen?

Aufgabe 2: Tiefensuche und topologisches Sortieren

Sie entwickeln ein Build-System für ein Softwareprojekt. Ein Modul darf erst übersetzt werden, wenn alle seine Abhängigkeiten bereits vorliegen. Die Abhängigkeiten zwischen den Modulen lauten:

Modul	hängt ab von
main	parser, codegen
parser	lexer, ast
codegen	ast, optimizer
optimizer	utils
ast	utils
lexer	(keine)
utils	(keine)

- Graph aufbauen:** Modellieren Sie die Abhängigkeiten als gerichteten azyklischen Graphen. Eine Kante von A nach B bedeutet: A hängt von B ab, B muss also vor A gebaut werden. Bauen Sie den Graphen mit `AdjacencyListGraph` auf.
- DFS ausführen:** Rufen Sie `dfs()` auf und geben Sie für jeden Knoten die Zeitmarken (aus `enter_map` und `leave_map`) aus.
- Topologische Sortierung:** Sortieren Sie die Knoten nach ihrer End-Zeitmarke in absteigender Reihenfolge. Das Ergebnis ist eine gültige Build-Reihenfolge. Überprüfen Sie, ob alle Abhängigkeiten eingehalten werden.
- Komplexität:** Die Tiefensuche hat die Zeitkomplexität $O(|V| + |E|)$. Wie viele Knoten und Kanten hat Ihr Graph? Erklären Sie anhand des konkreten Graphen, warum jeder Knoten und jede Kante genau einmal betrachtet wird.